



EGE UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

2023-2024 SPRING SEMESTER

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING
PROJECT-1 REPORT

DELIVERY DATE

19.05.2024

PREPARED BY

05230000391, Tuğra HEKİMOĞLU

05220001067, Aykut HACIOĞLU

Customer Data Class:

Variables:

```
1 package proje;
2
3 public class CustomerData {
4     private String name;
5     private String surname;
6     private String country;
7     private String city;
8     private String occupation;
```

name: Stores the customer's first name.

surname: Stores the customer's last name.

country: Stores the customer's country.

city: Stores the customer's city.

occupation: Stores the customer's occupation.

Constructors:

```
10 //Non parameter Constructor
11 public CustomerData() {
12 }
13
14 //Using the all variables Constructor
15 public CustomerData(String name, String surname, String country, String city, String occupation) {
16     this.name = name;
17     this.surname = surname;
18     this.country = country;
19     this.city = city;
20     this.occupation = occupation;
21 }
22
32 //Copy Constructor
33 public CustomerData(CustomerData other) {
34     this.name = other.name;
35     this.surname = other.surname;
36     this.country = other.country;
37     this.city = other.city;
38     this.occupation = other.occupation;
39 }
```

Non-parameter Constructor: Initializes a new instance of CustomerData with no initial values.

Parameterized Constructor: Initializes a new instance of CustomerData with specified values for all attributes.

Copy Constructor: Initializes a new instance of CustomerData by copying values from another CustomerData instance.

Getters and Setters:

```
40  [-] public String getName() {
41      return name;
42  }
43
44  [-] public void setName(String name) {
45      this.name = name;
46  }
47
48  [-] public String getSurname() {
49      return surname;
50  }
51
52  [-] public void setSurname(String surname) {
53      this.surname = surname;
54  }
55
56  [-] public String getCountry() {
57      return country;
58  }
59
60  [-] public void setCountry(String country) {
61      this.country = country;
62  }
63
64  [-] public String getCity() {
65      return city;
66  }
67
68  [-] public void setCity(String city) {
69      this.city = city;
70  }
71
72  [-] public String getOccupation() {
73      return occupation;
74  }
75
76  [-] public void setOccupation(String occupation) {
77      this.occupation = occupation;
78  }
```

Provide access to and modification of the attributes.

toString():

```
24      @Override
25      [-] public String toString() {
26          return "CustomerData{" + "name=" + name + ", surname=" + surname +
27              ", country=" + country + ", city=" + city + ", occupation=" + occupation + '}';
28      }
29
30
```

Returns a string representation of the CustomerData instance, including all attribute values.

Two Dimensional Array Class:

Variables:

```
5      public class TwoDimensionalArray {  
6          // Two-dimensional array to store ratings  
7          private int[][] ratings;  
8      }
```

ratings: A two-dimensional array of integers used to store ratings data.

Constructor:

```
9          // Initialize the ratings array with given rows and columns  
10     public TwoDimensionalArray(int rowCount, int colCount) {  
11         this.ratings = new int[rowCount][colCount];  
12     }
```

TwoDimensionalArray(int rowCount, int colCount): Initializes the ratings array with the specified number of rows and columns.

Methods:

```
14          // Return the ratings array  
15     public int[][] getRatings() {  
16         return ratings;  
17     }
```

getRatings(): Returns the ratings array.

```
19          // Set the ratings array to a new array  
20     public void setRatings(int[][] ratings) {  
21         this.ratings = ratings;  
22     }
```

setRatings(int[][] ratings): Sets the ratings array to a new two-dimensional array.

```

24 // Set a specific row with customer ID and ratings
25 public void setRatingsRow(int row, int[] rowData, int customerID) {
26     // Validate the row index and data length
27     if (row >= 0 && row < ratings.length && rowData.length + 1 == ratings[0].length) {
28         ratings[row][0] = customerID;
29         for (int i = 0; i < rowData.length; i++) {
30             ratings[row][i + 1] = rowData[i];
31         }
32     } else {
33         System.out.println("Invalid row index or row data length.");
34     }
35 }

```

setRatingsRow(int row, int[] rowData, int customerID): Sets a specific row in the ratings array, including a customer ID. It validates the row index and ensures the row data length matches the array's column length minus one.

```

37 // Print specified rows of the array
38 public static void printRatings(int[][] array, int rowCount) {
39     for (int i = 0; i < rowCount; i++) {
40         for (int j = 0; j < array[i].length; j++) {
41             System.out.print(array[i][j] + " ");
42         }
43         System.out.println();
44     }
45 }
46 }

```

printRatings(int[][] array, int rowCount): A static method to print the ratings array to the console, displaying each element in a formatted manner.

Customer Linked List Class:

Structure:

```
3 public class CustomerLinkedList {
4     static class Node {
5         // Represents a customer record in the linked list
6
7         private int customerNumber; // Unique customer number
8         private CustomerData customerData; // Customer information
9         private Node next; // Reference to the next node
10
11         // Constructor to initialize Node with customer number and data
12         public Node(int customerNumber, CustomerData customerData) {
13             this.customerNumber = customerNumber;
14             this.customerData = customerData;
15             this.next = null; // Initially, next is null
16         }
17     }
18
19     private Node head; // Reference to the first node
```

Node Class: Nested static class representing a single node in the linked list. Each node contains a customer number, customer data, and a reference to the next node. It has a constructor to initialize the node with customer number and data.

```
21 // Constructor to initialize an empty list
22 public CustomerLinkedList() {
23     this.head = null; // Initially empty
24 }
```

CustomerLinkedList Class: Main class representing the linked list itself and contains methods to manipulate the list. It has a reference to the first node (head) to maintain the structure of the linked list. Provides a constructor to initialize an empty list.

Methods:

```
26 // Add a new customer to the list
27 public void addCustomer(int customerNumber, CustomerData customerData) {
28     Node newNode = new Node(customerNumber, customerData); // Create new node
29
30     // Insert at the beginning if list is empty or new customer comes before head
31     if (head == null || customerNumber < head.customerNumber) {
32         newNode.next = head;
33         head = newNode;
34     } else {
35         Node current = head;
36         // Find appropriate position to insert new node
37         while (current.next != null && current.next.customerNumber < customerNumber) {
38             current = current.next;
39         }
40         // Insert the new node
41         newNode.next = current.next;
42         current.next = newNode;
43     }
44 }
```

addCustomer(int customerNumber, CustomerData customerData): Adds a new customer to the list. Creates a new node with the provided customer number and data. Inserts the node at the beginning if the list is empty or the new customer comes before the head. Otherwise, it finds the appropriate position based on customer number and inserts the node.

```
46 // Display the contents of the list
47 public void displayList() {
48     Node current = head;
49     while (current != null) {
50         System.out.println("Customer Number: " + current.customerNumber);
51         System.out.println("Customer Data: " + current.customerData.toString());
52         System.out.println(""); // Add an empty line for readability
53         current = current.next;
54     }
55 }
```

displayList(): Displays the contents of the list by traversing through each node. Prints the customer number and their data.

```

57 // Retrieve the country of a customer based on their customer number
58 public String getCountry(int customerNumber) {
59     Node current = head;
60     while (current != null) {
61         if (current.customerNumber == customerNumber) {
62             return current.customerData.getCountry();
63         }
64         current = current.next;
65     }
66     return null; // Return null if not found
67 }

```

getCountry(int customerNumber): Retrieves the country of a customer based on their customer number. Searches through the list for the node with the provided customer number and returns the country from the associated customer data.

```

69 // Retrieve the occupation of a customer based on their customer number
70 public String getOccupation(int customerNumber) {
71     Node current = head;
72     while (current != null) {
73         if (current.customerNumber == customerNumber) {
74             return current.customerData.getOccupation();
75         }
76         current = current.next;
77     }
78     return null; // Return null if not found
79 }
80 }

```

getOccupation(int customerNumber): Retrieves the occupation of a customer based on their customer number. Searches through the list for the node with the provided customer number and returns the occupation from the associated customer data.

Main Class:

Main Method:

```
8 public static void main(String[] args) {
9     // Initialize variables
10    String productCount = null;
11    String[] productNames = null;
12    int row = 0;
13    int numCustomers = 0;
14    int customerNumber = 0;
15    String[] items;
16    TwoDimensionalArray array = null;
17    CustomerLinkedList customerList = new CustomerLinkedList();
18    Scanner scanner = new Scanner(System.in);
19    int showMenu = 0;
20
21    // Display menu options
22    System.out.println("""
23        1)Reading the data from the file and creating the relevant data structures.
24        2)Entering new customer information via keyboard.
25        3)Calculating and printing the average rating score for each product.
26        4)Calculating and printing the average rating score obtained from only customers whose country is "Turkey" for each product.
27        5)Calculating and printing the average rating score obtained from only customers whose country is not "Turkey" for each product.
28        6)Calculating and printing the average rating score obtained from only customers whose profession is "Doctor" for each product.
29        7)Printing the linked list of customer information from the beginning to the end.
30        8)Printing the two-dimensional array to the screen.
31        9)Exit.""");
32
33    // Try to read the product names from the file
34    try {
35        Scanner scan = new Scanner(new FileInputStream("File.txt"));
36        if (scan.hasNextLine()) {
37            String firstLine = scan.nextLine();
38            String[] parts = firstLine.split(",");
39            productNames = new String[parts.length - 1];
40
41            // Extract product names from the first line of the file
42            for (int i = 1; i < parts.length; i++) {
43                productNames[i - 1] = parts[i].trim();
44            }
45        }
46    } catch (FileNotFoundException e) {
47        System.out.println("File could not be read: " + e.getMessage());
48    }
```

Initializes variables, displays a menu with options, reads product names from "File.txt", and enters a loop to handle user menu selections. Depending on the selection, it can read data from a file, input new customer information, calculate and print various average rating scores, display the customer linked list, print the two-dimensional array, or exit the program.

Conditions:

```
50 // Main menu loop
51 while (showMenu > 0 || showMenu <= 9) {
52     System.out.println("Select a process 1 to 9.");
53     showMenu = scanner.nextInt();
54
55     if (showMenu == 1) {
56         // Reading data from file and creating data structures
57         Scanner scanner1 = null;
58
59         try {
60             scanner1 = new Scanner(new FileInputStream("File.txt"));
61             if (scanner1.hasNextLine()) {
62                 String line = scanner1.nextLine();
63                 items = line.split(",");
64                 productCount = items[0];
65                 array = new TwoDimensionalArray(200, Integer.parseInt(productCount) + 1);
66             }
67             int lineCount = 1;
68
69             // Read customer and rating data from the file
70             while (scanner1.hasNextLine()) {
71                 lineCount++;
72                 String line = scanner1.nextLine();
73                 items = line.split(",");
74
75                 if (lineCount % 2 == 0) {
76                     // Processing customer data
77                     customerNumber = Integer.parseInt(items[0]);
78                     String name = items[1];
79                     String surname = items[2];
80                     String country = items[3];
81                     String city = items[4];
82                     String occupation = items[5];
83
84                     // Create and add customer data to the list
85                     CustomerData customerData = new CustomerData(name, surname, country, city, occupation);
86                     customerList.addCustomer(customerNumber, customerData);
87                     numCustomers++;
88                 } else {
89                     // Processing ratings data
90                     row = lineCount / 2 - 1;
91                     int[] ratingsRow = new int[Integer.parseInt(productCount)];
92                     for (int i = 0; i < items.length; i++) {
93                         ratingsRow[i] = Integer.parseInt(items[i]);
94                     }
95                     array.setRatingsRow(row, ratingsRow, customerNumber);
96                 }
97             }
98             catch (FileNotFoundException e) {
99                 System.out.println("File could not be read: " + e.getMessage());
100             }
101         }
102
103         if (showMenu == 2) {
104             // Enter new customer information
105             if (numCustomers + 1 > 200) {
106                 System.out.println("Number of customers cannot be more than 200.");
107                 return;
108             } else {
109                 numCustomers++;
110                 enterNewCustomer(scanner, customerList, productCount, array, numCustomers);
111             }
112         }
113
114         if (showMenu == 3) {
115             // Calculate and print the average rating for each product
116             calculateAverage(array.getRatings(), productNames);
117         }
118
119         if (showMenu == 4) {
120             // Calculate and print the average rating for each product by customers from Turkey
121             calculateAverageByCountry(array.getRatings(), customerList, productNames, "Turkey");
122         }
123
124         if (showMenu == 5) {
125             // Calculate and print the average rating for each product by customers not from Turkey
126             calculateAverageByCountry(array.getRatings(), customerList, productNames, "not Turkey");
127         }
128     }
129 }
```

```

129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149

if (showMenu == 6) {
    // Calculate and print the average rating for each product by customers who are doctors
    calculateAverageByOccupation(array.getRatings(), customerList, productNames, "Doctor");
}

if (showMenu == 7) {
    // Print the linked list of customer information
    customerList.displayList();
}

if (showMenu == 8) {
    // Print the two-dimensional array
    int[][] ratings = array.getRatings();
    array.printRatings(ratings, numCustomers);
}

if (showMenu == 9) {
    // Exit the program
    System.out.println("Exiting...");
    System.exit(0);
}

```

Option 1 - Read Data from File:

- Reads product names and customer information from "File.txt" to initialize data structures.
- Processes each line of the file to extract customer data and ratings.
- Creates a TwoDimensionalArray to store ratings and a linked list to store customer information.

Option 2 - Enter New Customer Information:

- Allows the input of new customer information via the keyboard.
- Checks if the maximum customer limit (200) is reached to prevent overflow.
- Prompts the user for customer details and product ratings.
- Predicts the rating for the last product using existing ratings and updates the data structures accordingly.

Options 3-6 - Calculate Average Ratings:

- Calculate and print various average rating scores based on different criteria:
- **Option 3:** Overall average rating for each product.
- **Option 4:** Average rating for each product by customers from Turkey.
- **Option 5:** Average rating for each product by customers not from Turkey.
- **Option 6:** Average rating for each product by customers with a specified occupation (e.g., doctors).

Option 7 - Print Linked List of Customer Information:

- Prints the linked list of customer information stored in memory.
- Traverses the linked list and prints details of each customer.

Option 8 - Print Two-Dimensional Array of Ratings:

- Prints the two-dimensional array of ratings to the screen.
- Displays the ratings in a structured format, showing customer ratings for each product.

Option 9 - Exit the Program:

- Allows the user to exit the program gracefully.

Methods:

```

153 // Method to enter new customer information
154 private static void enterNewCustomer(Scanner scanner, CustomerLinkedList customerList, String productCount, TwoDimensionalArray array, int numCustomers) {
155     System.out.println("Enter customer information:");
156     System.out.print("Enter the customer number: ");
157     int customerNumber = scanner.nextInt();
158     System.out.print("Enter the customer's name: ");
159     String name = scanner.next();
160     System.out.print("Enter the customer's surname: ");
161     String surname = scanner.next();
162     System.out.print("Enter the country where customer lives: ");
163     String country = scanner.next();
164     System.out.print("Enter the city where customer lives: ");
165     String city = scanner.next();
166     System.out.print("Enter the customer's occupation: ");
167     String occupation = scanner.next();
168
169     // Create a new customer data object and add it to the list
170     CustomerData newCustomer = new CustomerData(name, surname, country, city, occupation);
171     customerList.addCustomer(customerNumber, newCustomer);
172     int numProducts = Integer.parseInt(productCount);
173     int[] ratings = new int[numProducts - 1];
174
175     // Get ratings for each product
176     for (int i = 0; i < numProducts - 1; i++) {
177         System.out.print("Points for product " + (char) ('A' + i) + "(1-5): ");
178         ratings[i] = scanner.nextInt();
179         while (ratings[i] < 1 || ratings[i] > 5) {
180             System.out.print("Wrong input! Input should be (1-5), points for product " + (char) ('A' + i) + ": ");
181             ratings[i] = scanner.nextInt();
182         }
183     }
184
185     // Predict the rating for the last product
186     int predictedRating = predictRating(array.getRatings(), ratings, numProducts - 1);
187     System.out.println("Predicted rating: " + predictedRating);
188     int[] completeRatings = new int[numProducts];
189     System.arraycopy(ratings, 0, completeRatings, 0, ratings.length);
190     completeRatings[numProducts - 1] = predictedRating;
191     array.setRatingsRow(numCustomers - 1, completeRatings, customerNumber);
192     System.out.println("New customer information added successfully.");
193 }

```

enterNewCustomer(Scanner scanner, CustomerLinkedList customerList, String productCount, TwoDimensionalArray array, int numCustomers):

This method is used to enter new customer information and ratings via the keyboard. Prompts the user to enter customer details (number, name, surname, country, city, occupation), and product ratings. It then predicts the rating for the last product using the predictRating method, updates the ratings array with the new customer's information, and confirms the addition.

```

195 // Method to predict rating for the last product based on existing ratings
196 private static int predictRating(int[][] ratings, int[] newRatings, int lastProductIndex) {
197     if (ratings == null || ratings.length == 0 || newRatings == null || newRatings.length == 0) {
198         throw new IllegalArgumentException("Ratings data or new ratings data is invalid.");
199     }
200     int minSimilarity = Integer.MAX_VALUE;
201     int predictedRating = 0;
202     int similarCustomerCount = 0;
203     int similarCustomerTotalRating = 0;
204
205     // Calculate similarity between new ratings and existing ratings
206     for (int i = 0; i < ratings.length; i++) {
207         if (ratings[i].length <= lastProductIndex + 1) {
208             continue;
209         }
210         int similarity = 0;
211         for (int j = 1; j <= lastProductIndex; j++) {
212             similarity += Math.abs(newRatings[j - 1] - ratings[i][j]);
213         }
214
215         // Update predicted rating based on most similar customer
216         if (similarity < minSimilarity) {
217             minSimilarity = similarity;
218             predictedRating = ratings[i][lastProductIndex + 1];
219             similarCustomerCount = 1;
220             similarCustomerTotalRating = ratings[i][lastProductIndex + 1];
221         } else if (similarity == minSimilarity) {
222             similarCustomerTotalRating += ratings[i][lastProductIndex + 1];
223             similarCustomerCount++;
224         }
225     }
226
227     // If there are multiple similar customers, average their ratings
228     if (similarCustomerCount > 1) {
229         predictedRating = similarCustomerTotalRating / similarCustomerCount;
230     }
231
232     return predictedRating;
233 }

```

predictRating(int[][] ratings, int[] newRatings, int lastProductIndex):

This method predicts the rating for the last product based on the new ratings provided. Calculates similarity between the new ratings and existing customer ratings, finds the most similar customer(s), and determines the predicted rating for the last product. If multiple similar customers are found, their ratings are averaged.

```

235 // Method to calculate and print the average rating for each product
236 private static void calculateAverage(int[][] ratings, String[] productNames) {
237     int numCustomers = ratings.length;
238     int numProducts = ratings[0].length - 1;
239
240     for (int j = 1; j <= numProducts; j++) {
241         int totalRating = 0;
242         int ratingCount = 0;
243
244         // Sum ratings for each product
245         for (int i = 0; i < numCustomers; i++) {
246             if (ratings[i][j] != 0) {
247                 totalRating += ratings[i][j];
248                 ratingCount++;
249             }
250         }
251
252         // Calculate and print average rating
253         double averageRating = (double) totalRating / ratingCount;
254         System.out.println("Average rating for product " + productNames[j - 1] + ": " + averageRating);
255     }
256 }

```

calculateAverage(int[][] ratings, String[] productNames):

This method calculates and prints the average rating score for each product. Iterates through each product, sums the ratings, counts the number of ratings, computes the average rating, and prints the average rating for each product.

```

258 // Method to calculate and print the average rating for each product by country
259 private static void calculateAverageByCountry(int[][] ratings, CustomerLinkedList customerList, String[] productNames, String country) {
260     int numCustomers = ratings.length;
261     int numProducts = ratings[0].length - 1;
262
263     for (int j = 1; j <= numProducts; j++) {
264         int totalRating = 0;
265         int ratingCount = 0;
266
267         // Sum ratings for each product by country
268         for (int i = 0; i < numCustomers; i++) {
269             if (ratings[i][j] != 0 && (country.equals("not Turkey") ? !customerList.getCountry(ratings[i][0]).equals("Turkey") :
270                 customerList.getCountry(ratings[i][0]).equals("Turkey"))) {
271                 totalRating += ratings[i][j];
272                 ratingCount++;
273             }
274         }
275
276         // Calculate and print average rating by country
277         double averageRating = (double) totalRating / ratingCount;
278         System.out.println("Average rating for product " + productNames[j - 1] +
279             " (" + (country.equals("not Turkey") ? "Not Turkey" : "Turkey") + "): " + averageRating);
280     }
281 }
282

```

calculateAverageByCountry(int[][] ratings, CustomerLinkedList customerList, String[] productNames, String country):

This method calculates and prints the average rating score for each product by customers from a specified country. Filters the ratings based on the specified country (or "not Turkey"), sums the relevant ratings, counts the ratings, computes the average, and prints the average rating for each product.

```

282 private static void calculateAverageByOccupation(int[][] ratings, CustomerLinkedList customerList, String[] productNames, String occupation)
283 {
284     int numCustomers = ratings.length;
285     int numProducts = ratings[0].length - 1;
286
287     for (int j = 1; j <= numProducts; j++) {
288         int totalRating = 0;
289         int ratingCount = 0;
290
291         // Sum ratings for each product by occupation
292         for (int i = 0; i < numCustomers; i++) {
293             if (ratings[i][j] != 0 && customerList.getOccupation(ratings[i][0]).equals(occupation)) {
294                 totalRating += ratings[i][j];
295                 ratingCount++;
296             }
297         }
298
299         // Calculate and print average rating by occupation
300         double averageRating = (double) totalRating / ratingCount;
301         System.out.println("Average rating for product " + productNames[j - 1] + " (Occupation: " + occupation + "): " + averageRating);
302     }
}

```

calculateAverageByOccupation(int[][] ratings, CustomerLinkedList customerList, String[] productNames, String occupation):

This method calculates and prints the average rating score for each product by customers of a specified occupation. Filters the ratings based on the specified occupation, sums the relevant ratings, counts the ratings, computes the average, and prints the average rating for each product.